SlickScatter: Retrieve WiFi Backscatter Signal from Unknown Interference

Shanyue Wang[1], Feiyu Han[1], Yubo Yan[1,3], Yuxin Ding[1], Panlong Yang[2], Xiang-Yang Li[1*]
[1]School of Computer Science and Technology, University of Science and Technology of China, Hefei, China
[2]School of Computer Science, Nanjing University of Information Science and Technology, Nanjing, China
[3]Deqing Alpha Innovation Institute, Huzhou, China
Email: wsy1998@mail.ustc.edu.cn, fyhan@mail.ustc.edu.cn, yuboyan@ustc.edu.cn,
yx_ding@mail.ustc.edu.cn, plyang@ustc.edu.cn, xiangyangli@ustc.edu.cn

***Abstract*—WiFi backscatter communication demonstrates significant potential for the upcoming era of low-power wireless networks. Nevertheless, due to the low-power requirement of backscatter tags, there are limitations in their capacity to eliminate conflicts, posing a significant challenge for WiFi backscatter communication in environments with ambient interference. To address that, we introduce SlickScatter, an interference-insensitive WiFi backscatter system that can retrieve WiFi backscatter signals even in the presence of unknown ambient interference. The core strategy of SlickScatter involves designating a portion of the tag data symbols as pilot symbols. This approach enables the detection of uninterfered subcarriers and the estimation of channel state information and phase errors for all symbols within a packet, facilitating the demodulation of packets affected by interference. We have prototyped and evaluated SlickScatter with 802.11g OFDM WiFi signals, demonstrating its robustness and effectiveness against unknown ambient interference. Compared with a state-of-the-art solution, SlickScatter significantly decreases the frame error rate by 50% and improves the throughput by $1.96\times$ at a distance of 12 m.**

***Index Terms*—Backscatter, WiFi, Wireless Communication, Internet of Things**

I. INTRODUCTION

In recent years, ambient backscatter technologies have gained increasing attention for their potential to enable near-zero-power communication for billions of IoT devices [1]–[6]. They leverage ambient signals as wireless carriers, broadening the range of excitation sources beyond dedicated readers to include various ambient signals [5], [7]. Among them, WiFi backscatter communication technology has been widely studied due to the widespread availability of WiFi signals and the abundance of WiFi devices. For instance, HitchHike [8] introduces a codeword translation technique that extracts the tag data only through demodulating WiFi packets. This simple and practical approach enables backscatter systems to be deployed seamlessly on conventional 802.11b WiFi devices, Later works [9], [10] extend this technology to 802.11g/n protocols, which are based on the advanced orthogonal frequency-division multiplexing (OFDM) modulation scheme. In addition, some recent works have optimized it in terms of system performance [11]–[15] and system architecture [16]–[18].

However, current WiFi backscatter systems still face challenges in moving towards practical deployment. A significant issue is the coexistence of various ambient signals, such as Bluetooth and ZigBee, which share the same frequency band with WiFi and disrupt WiFi backscatter communication. Unfortunately, existing backscatter tags lack the capability for conflict avoidance and error retransmission due to their extreme power consumption design. This limitation stems from the absence of components such as wideband and high-precision analog-to-digital converters (ADCs), which are essential for demodulating request packets from various ambient sources. Previous studies [9], [10], [18]–[20] directly reuse ambient WiFi signals to transmit data, which are particularly susceptible to interference from other signals in the same band. The broad signal bandwidth of WiFi backscatter (20 MHz) further increases the likelihood of interference. The inability of existing WiFi backscatter systems to demodulate interfered signals stems from a lack of foreknowledge about the interference. For instance, the double-RX-based backscatter systems [8]–[11], [19] rely on comparing received packet data with known TX signals to extract tag phase information. However, this approach fails when backscatter packets are disrupted by unidentified ambient signals, making it difficult for receivers to differentiate between tag and interference phases. Some works utilize the specific property of the WiFi packet like the zero-subcarrier [17] and extra CSI [13] to transmit data, but the unknown ambient signal interference signal would also destroy such property, leading to demodulation failures. Thus, a crucial problem emerges: *how to accurately demodulate WiFi backscatter signals from unknown ambient interference?*

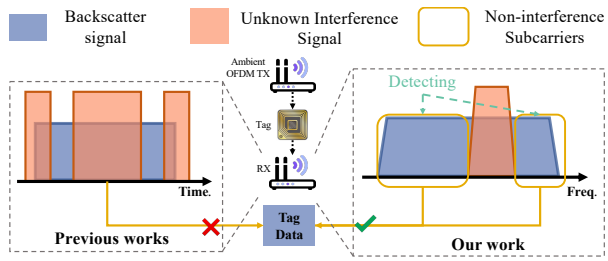


Fig. 1: SlickScatter mainly works in frequency domain. It can detect the uninterfered subcarriers of backscatter packets and use them to demodulate the tag data.

* Yubo Yan and Xiang-Yang Li are corresponding authors.

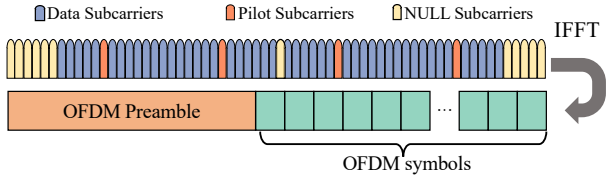


Fig. 2: OFDM-based WiFi system.

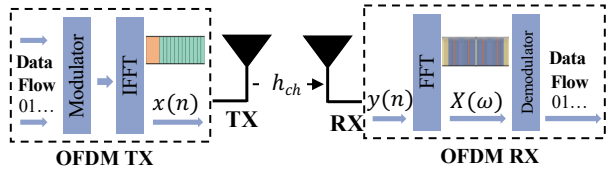


Fig. 3: The channel model of typical OFDM-based WiFi system.

To solve this problem, we propose SlickScatter, a novel WiFi backscatter system that can demodulate tag data from the backscatter signal disturbed by unknown interference sources. Leveraging the OFDM-based WiFi protocol [21], SlickScatter enables the accurate demodulation of the interfered backscatter signals by identifying even one uninterfered subcarrier in a symbol, as illustrated in Fig. 1. Grounded in the aforementioned property, we should pose two primary challenges in demodulating interfered backscatter packets:

1) How to identify the uninterfered subcarriers in a backscatter packet that has been disrupted by dynamic interference? When interference occurs, some subcarriers become compromised. Identifying these interfered subcarriers presents two significant challenges for the receiver. Firstly, interference from ambient signals manifests unpredictably across both time and spectrum. Within a single packet period, interference can shift across different subcarriers. And the receiver lacks the capability to foresee these interference occurrences. Secondly, a fundamental aspect of tag data modulation involves introducing phase errors into the ambient WiFi packet. This complicates the receiver's task during demodulation, making it challenging to discern whether errors stem from the tag data or the interference.

2) How to demodulate tag data from uninterfered subcarriers independently of previous results? To demodulate backscatter data using uninterfered subcarriers in one symbol, it's crucial to precisely obtain common phase error (CPE) and channel state information (CSI) [11], [17], [21] of current symbol. However, estimating these two parameters needs to be based on the results calculated previously. For CPE estimation, prior methods [17], [22] suggest a recursive algorithm, requiring the previous symbol's CPE for the current symbol's estimation. However, this approach faces challenges as the frequency band of interference signals may change over time. Consequently, if a subcarrier is interfered with at any symbol, its CPE estimation for that symbol fails, rendering it unusable for future CPE estimations and demodulation, even if it becomes uninterfered later. Furthermore, interference can also affect the accuracy of CSI estimation of subcarriers, particularly if the backscatter packet's long training field (LTF) is affected. Without precise CPE and CSI, the receiver is unable to accurately demodulate the tag's true phase.

To address these problems, we employ a method that incorporates pilot bits with fixed bit sequences into the tag data. The corresponding symbols in backscatter packets aid in identifying their uninterfered subcarriers while also acquiring the CPE and CSI for all packet symbols. To achieve this, we introduce a metric defined as Related Channel State Information (RCSI) for each subcarrier within the pilot symbols. By analyzing the variance of RCSI, we can effectively distinguish between subcarriers that are interfered with and those that are not. We then devise a CPE fitting method that capitalizes on the linear characteristic of CPE, enabling us to estimate both the CPE and CSI for the entirety of the backscatter packet. Additionally, we introduce a multi-subcarrier voting algorithm for decoding tag data. By integrating these strategies, we ensure accurate demodulation of tag data despite the presence of unknown ambient signal interference.

Our **contributions** are summarized as follows:

- As far as our knowledge, SlickScatter is the first interference-insensitive WiFi backscatter system that can demodulate disrupted WiFi backscatter signals from unknown ambient interference.
- To deal with the interference, we insert pilot symbols in the backscatter WiFi packet to detect the uninterfered subcarriers and estimate the channel state. Based on that, we propose uninterfered subcarrier detection, phase error fitting, and multi-subcarrier voting algorithms to demodulate tag data of all symbols.
- We propose SlickScatter on 802.11g WiFi protocol with software-defined radios. The experiment results show that SlickScatter can keep the BER below 10^{-2} level in the office environment. It can decrease 50% frame error rate and achieve $1.96\times$ goodput than RapidRider in a distance of 12m.

II. PRELIMINARY

A. OFDM based WiFi

In this paper, we concentrate solely on the OFDM-based WiFi backscatter system, which is fundamental to the IEEE 802.11a/g/n/ac protocols. We choose 802.11g protocol for example. As illustrated in Fig. 2, the standard structures the signal with 64 subcarriers in the frequency domain, where the data symbols and the pilot symbols have payloads. During transmission, the signal undergoes conversion to the time domain via Inverse Fast Fourier Transform (IFFT). The time-domain signal's structure is divided into two main parts: the OFDM preamble and the data symbols. The preamble is critical for boundary detection and CSI estimation, while the data symbols carry the transmitted information. Each symbol consists of 64 subcarriers in the frequency domain.

Channel model. The channel model for the 802.11g standard OFDM WiFi is depicted in Fig. 3. The received frequency

domain signal on each subcarrier in this system is mathematically represented as:

$$y(n) = h_{ch} \cdot x(n) \cdot e^{j\phi_{ce}(n)} + v(n)$$

where $x(n)$ represents the n-th symbol transmitted by the transmitter (TX) for a specific subcarrier. The term h_{ch}, a complex number, signifies CSI, while ϕ_{ce} reflects CPE due to the carrier-frequency offset (CFO). The variable v denotes noise. It's assumed that for each subcarrier, the CSI h_{ch} remains unchanged during a frame's transmission, enabling the utilization of the predefined preamble for the estimation of h_{ch}.

B. OFDM WiFi backscatter

The prevailing structure OFDM backscatter system involves two receivers (RXs) [9], [20]: one for receiving packets from the transmitter, and the other for capturing backscattered packets on a non-interfering channel. These systems often use symbol-level demodulation based on codeword translation, where tags utilize Phase Shift Keying (PSK) to encode data onto TX packet symbols. The crucial aspect of this method is that tag translates the OFDM symbol from one valid codeword to another, which allows the receiver to directly demodulate the WiFi packet with the errors introduced by the tag. This is achieved by comparing signals from both RXs to extract tag data from the erroneous packet data. However, if the unknown interference happens, RX cannot distinguish whether the phases is introduced by the interference sources or tag, which makes the demodulation of the backscatter signal difficult.

Backscatter channel model. The channel model for OFDM backscatter communication is illustrated in Fig. 4. In this model, TX sends out a WiFi signal stream that travels through the channel between the TX and the tag (TX-tag channel). Simultaneously, to avoid self-interference, the tag modulates this signal using a square wave, shifting it to a different WiFi channel (tag-RX channel) [8]. Consequently, the backscatter signal, denoted as $y_b(n)$ received by antenna can be represented by the following:

$$y_b(n) = h_R \cdot x(n) \cdot e^{j\phi_t(n)} \cdot h_T \cdot e^{j\phi_{ce}(n)} + v(n),$$

where h_R and h_T represent the CSI for the TX-tag and tag-RX links, respectively, and $\phi_t(n)$ denotes the tag's phase shift, which is determined by the chosen demodulating methods. Within the framework of a single-input single-output (SISO) 802.11g protocol, these two CSIs can be combined into a single composite channel h_{BS}. Consequently, the received tag signal at the receiver can be expressed as:

$$y_b(n) = h_{BS} \cdot x(n) \cdot e^{j\phi_t(n)} \cdot e^{j\phi_{ce}(n)} + v(n), \qquad (1)$$

where h_{BS} and ϕ_{ce} are readily estimable using standard WiFi techniques [21] with long training fields (LTFs), and $x(n)$ is ascertainable through the dual RX structure [11]. Therefore, in the absence of interference, the receiver can readily demodulate the tag signal.

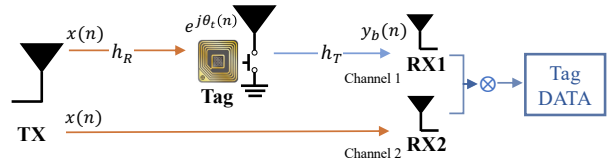


Fig. 4: The channel model of WiFi backscatter system.

Signal type	Bandwidth
ZigBee	2 MHz
Bluetooth/BLE	1 MHz
WiFi(802.11b,g,n,ac,ax)	20 MHz

TABLE I: The bandwidth of different kinds of signals in WiFi 2.4 GHz, which can cause interference to backscatter signal.

C. The impact of interference signal.

If a subcarrier encounters interference, its expression can be modelled as follows:

$$y_{bi}(n) = h_{BS} \cdot x(n) \cdot e^{j\phi_t(n)} \cdot e^{j\phi_{ce}(n)} \cdot \eta(n) + v(n) \qquad (2)$$

Here, $\eta(n)$ represents the interference phase on the n-th subcarrier, whose value is unknown. Assuming we possess necessary parameters for demodulation, such as h_{BS}, ϕ_{ce}, and $x(n)$, the outcome for the tag phase becomes $\eta(n) \cdot e^{j\phi_t(n)}$. Typically, the interference signal's strength significantly exceeds that of the tag signal, causing the tag phase to be obscured by the interference phase. Consequently, demodulating the tag data from interfered subcarriers becomes impractical, necessitating the identification of uninterfered subcarriers for successful demodulation.

Our approach is predicated on the observation that interferences rarely span the entire bandwidth of the backscatter signal. For instance, as detailed in Table I, within the 2.4G WiFi band, various signals coexist, each with distinct bandwidths. However, the bandwidths of Bluetooth and Zigbee signals are considerably narrower than that of the backscatter signal. Interference across the entire bandwidth of the backscatter signal typically occurs only when the interfering source is a WiFi signal operating on the same channel concurrently, a relatively rare occurrence. Therefore, identifying and utilizing uninterfered subcarriers for demodulation is a feasible strategy.

III. System Overview

As shown in Fig. 5, our system consists of three main components: an ambient WiFi TX, two WiFi RXs, and a backscatter tag:

- **Ambient WiFi TX:** The ambient WiFi is a standard WiFi transmitter, which is used to provide the excitation signal for the backscatter tag. It emits the OFDM-based WiFi signal protocol 802.11g.
- **Backscatter tag:** The backscatter tag, built from commercial off-the-shelf (COTS) components, embeds its data into ambient WiFi packets and backscatter the signal to the WiFi receiver.

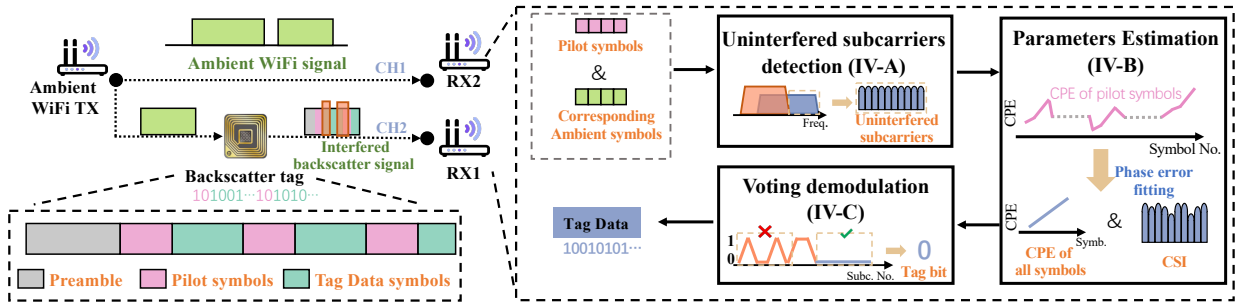


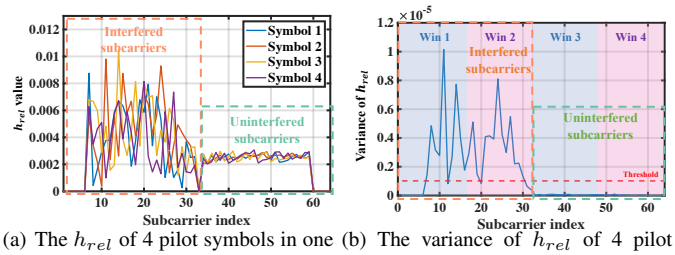
Fig. 5: System Overview. The backscatter tag encodes its data onto ambient WiFi signals using packets with embedded pilot bits. The receiver identifies clean subcarriers through pilot symbols, calculates CPE and CSI of all symbols, and demodulates the data from packets under unknown interference.

- **RXs:** We utilize two WiFi receivers to independently capture the ambient WiFi signal and the backscattered WiFi signal on two distinct channels. They can easily interact with each other, and we choose one of them to demodulate the tag data. Furthermore, we can demodulate the backscatter data even if it is interfered with by the unknown ambient signals.

Our approach leverages the passive reflection characteristic of the backscatter tag, which applies the same phase shift across all subcarriers within each symbol. Only by identifying subcarriers unaffected by ambient signal interference, we can successfully demodulate the backscatter data. This is achieved by incorporating pilot symbols into the backscatter packet structure. Utilizing these pilots, the receiver can identify their uninterfered subcarriers and effectively determine the CSI, phase errors.

In our backscatter tag system, we initially shift the received ambient WiFi signal to a different channel using the same approach as in previous studies [8]–[10] to prevent self-interference. Concurrently, the tag modulates its data onto each symbol through codeword translation as proposed in [8]. This process involves computing the backscatter codeword C_b from the original TX signal codeword C_o and the tag's square wave phase C_t, using the operation $C_b = C_o \oplus C_t$. The tag employs Binary Phase Shift Keying (BPSK) for data modulation, where a square wave phase of '0' represents data '0' and a phase of 'π' represents data '1'. Within SlickScatter, we embed pilot bits with a known bit sequence into the tag data, selecting N_p adjacent bits as a *pilot group*, separated by N_d data bits. In the backscatter packet, pilot bits and data bits correspond to pilot symbols and data symbols, respectively.

In the receiver, the initial step involves demodulating and recovering the ambient WiFi signal, which serves as a reference for further processes. Using this reference, along with the backscatter signal's pilot symbols, the receiver can identify their uninterfered subcarriers. Subsequently, the receiver employs a phase error fitting algorithm to accurately estimate the CPEs and Channel CSI for all symbols within the backscatter packet. The final step involves demodulating the backscatter data using a multi-subcarrier voting demodulation algorithm, effectively mitigating the effects of any interference band changes on data symbols.



(a) The h_{rel} of 4 pilot symbols in one group (b) The variance of h_{rel} of 4 pilot symbols in one group

Fig. 6: The diagram of uninterfered subcarriers detection.

IV. Design

In this section, we introduce the details of the receiver's design for extracting the tag data from the interfered packets. In §II-B, we have introduced the demodulating procedure of **uninterfered subcarrier** need to estimate two parameters: **CSI** and **CPEs**. To do this, we first detect the uninterfered subcarriers of the pilot symbols. Then, we use these subcarriers to estimate the phase errors and CSI of all symbols and finally demodulate tag data.

A. Uninterfered subcarrier detection

In the receiver, we access all pilot symbol subcarriers from the backscatter packet. The uninterfered subcarrier is described by Equ. 1, while the interfered subcarrier follows Equ. 2. To demodulate tag data from subcarriers affected by interference, we develop an algorithm that identifies uninterfered subcarriers to facilitate accurate demodulation.

Our algorithm operates on the principle that, over short symbol durations, both CSI and phase errors remain relatively stable. Thus, we define RCSI h_{rel} of the ith symbol in one pilot group as :

$$h_{rel} = y_r(i) \cdot \overline{x(i) \cdot e^{j\phi_t(i)}}, i = 1, ..., N_p.$$

Here, $y_r(i)$ represents a subcarrier from the point group's i^{th} symbol, which could be either interfered or uninterfered. The notation $\overline{()}$ denotes the conjugation of a complex value, $\phi_t(i)$ symbolizes the phase of tag data in pilot symbols that are known, and N_p is the count of pilot symbols in one group. For uninterfered subcarriers, we derive: $h_{rel} =$

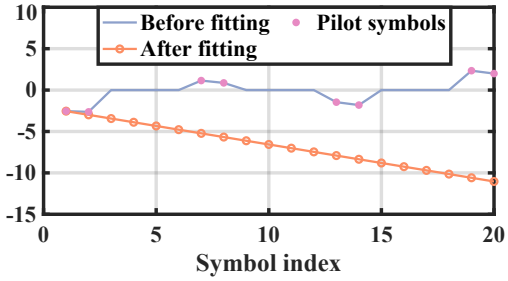


Fig. 7: The results of CPE fitting algorithm.

$h_{BS} \cdot e^{j\phi_{ce}(i)} + v(i)$. In contrast, for interfered subcarriers: $h_{rel} = h_{BS} \cdot e^{j\phi_{ce}(i)} \cdot \eta(i) + v(i)$. Typically, the interference parameter $\eta(i)$ exhibits significant fluctuations among pilot symbols, whereas the backscatter channel coefficient h_{BS} and the phase correction $e^{j\phi_{ce}(i)}$ remain stable, with noise $v(i)$ having minimal effect.

In Fig. 6(a), we present an example showing the h_{rel} values for each subcarrier across four pilot symbols within a single pilot group. It's observable that h_{rel} for interfered subcarriers exhibits significant fluctuations between the two pilot symbols, whereas, for uninterfered subcarriers, the values remain consistent. Further analysis is demonstrated in Fig. 6(b), where we plot the variance of h_{rel} for each subcarrier. This clearly illustrates a substantial difference between the variances of interfered and uninterfered subcarriers.

Leveraging this insight, we introduce a window detection algorithm aimed at identifying uninterfered subcarriers. We specify a window bandwidth of 5MHz, encompassing 16 subcarriers, and proceed to slide this window in steps of 16 subcarriers to assess if the variance values fall below a pre-defined threshold. Subcarriers within a window displaying variance values under this threshold are classified as uninterfered, indicating their suitability for further processing.

A key problem with this algorithm is setting the threshold. In our system, we initially define the threshold as $0.2 * M$ (detailed in §VI-A2), where M represents the highest variance value observed. This threshold effectively identifies uninterfered subcarriers, especially when only a portion of the subcarriers are affected by interference, given that the h_{rel} variance for interfered subcarriers significantly exceeds that of the uninterfered ones. To differentiate situations where all subcarriers are either interfered with or free from interference, we introduce an additional fixed threshold. Our experiments indicate that a threshold of 10^{-7} efficiently distinguishes between these two scenarios (§VI-A2). When most variance values fall well below this threshold, it suggests that all subcarriers are clear of interference. Conversely, if they exceed this threshold, it implies total interference, rendering our system ineffective. However, such total interference situations are exceedingly rare in our experimental observations.

B. Parameters estimation

Having identified the uninterfered subcarriers within the pilot symbols, we leverage the consistent behavior of phase errors and CSI across the entire packet to estimate these parameters for the data symbols. The process begins with estimating and compensating for phase errors in all symbols, followed by the estimation of CSI. Next, we detail the method for estimating these crucial parameters in the data symbols.

Phase errors estimation. Previous research [17], [22] introduced a recursive algorithm to estimate phase errors symbol by symbol. However, this approach falls short when dealing with backscatter signals affected by unpredictable interference from other sources. In such scenarios, incorrect phase error estimates for a given symbol can lead to accumulating errors in the following symbols. To address this issue, we analyze the underlying causes of phase error, which primarily include CFO and sampling frequency offset (SFO). CFO results from the discrepancy in frequency between the transmitter (TX) and receiver (RX), affecting all subcarriers uniformly and remaining constant throughout the packet transmission. The offset leads to a linear change in phase error across symbol indices, described by:

$$\phi_{ce}(n) = \phi_{ce}(0) + f_c \cdot n$$

,where the f_c is the CFO value, $\phi_{ce}(0)$ is the phase error of the first symbol. Our main purpose first is to estimate f_c of each subcarrier using linear fitting with the known parameters in pilot symbols.

We first reexpress the uninterfered subcarrier of the n-th symbol as:

$$\begin{aligned} y_b(n) &= h_{BS} \cdot x(n) \cdot e^{j\phi_t(n)} \cdot e^{j(\phi_{ce}(0)+f_c \cdot n)} + v(n) \\ &= h_{ref} \cdot x(n) \cdot e^{j\phi_t(n)} \cdot e^{jf_c \cdot n} + v(n), \end{aligned}$$

where $h_{ref} = h_{BS} \cdot e^{j\phi_{ce}(0)}$, named as reference channel state, is a consistent value across all symbols. Thus, the phase error of the n-th symbol can be expressed as:

$$\phi_{ce}(n) = \angle \frac{y_b(n)}{h_{ref}(n)x(n)e^{j\phi_t(n)}} \quad = f_c \cdot n - 2k\pi, k = 1, 2...,$$

$$\angle \frac{y_b(n)}{h_{ref}(n)x(n)e^{j\phi_t(n)}e^{jf_c t}} = f_c \cdot (n - t) - 2k\pi, k = 1, 2..., \tag{3}$$

where t is the first uninterfered pilot symbol index of this subcarrier.

However, there are two unknown quantities in this expression: f_c and k. A naive method is setting the interval of two pilot groups (N_d) to limit the difference of their phase errors below 2π, which can cancel the impact of k. However, in our system, the rate of phase error increase is faster than that of the traditional WiFi system, and this will significantly sacrifice the efficiency of the system. It is because we cannot perform compensation procedures like traditional OFDM WiFi systems [21], since the STFs and LTFs may be interfered with, introducing extra errors when compensation. To apply linear fitting for phase error estimation, we propose an algorithm to search for the correct value of k.

To search the value of k, we propose a searching algo-

rithm based on maximum likelihood estimation(MLE), which minimizes the Euclidean distance between the estimated phase error values of pilot symbols $\phi_{ce}(n)+2k\pi$ and the recalculated values $f_c \cdot n$:

$$f_c, k = \arg \min_{k=1,2,\ldots,T} \sum_{n \in P} \|\phi_{ce}(n) + 2k\pi, f_c \cdot n\|^2 \quad (4)$$

, where we can use linear fitting to estimate the f_c with the selected k value, P is the set of symbol index of pilot symbols, $\|\|$ is the Euclidean distance, and T is the upper boundary we set depending on the interval of two pilot groups and our experiment result. With the correct f_c value, we can estimate the phase errors of all symbols only if we know the phase error of any pilot symbol. The impact of SFO, which results from discrepancies in the sampling clock creating a linear relationship with the subcarrier index, is very weak and can be ignored in our system. Thus, we can choose the f_c of the subcarrier with h the highest number of uninterfered pilot symbols as the f_c of all subcarriers.

Fig. 7 illustrates the results of the phase error fitting algorithm, using $N_p = 2$ and $N_d = 4$ as an example for clearly demonstrating (the impact of pilot symbol number settings is analyzed in §VI-A). In this example, initially calculating the phase error of the pilot symbols via Equ. 3 yields CPE values within the range of $[-\pi, \pi]$. These values may not reflect the actual phase error, potentially deviating by $2k\pi$ from the actual value. Therefore, direct linear fitting using the pilot symbols' CPE values is impractical. By applying the search algorithm (Equ. 4), we can accurately compute the value of f_c and subsequently determine the correct CPE for all symbols of the current subcarrier.

CSI estimation. In the above analysis, what we need to estimate is not the true CSI h_{BS}, but the h_{ref}, which is also a consistent value across all symbols. It is worth mentioning that we need to estimate h_{ref} as many subcarriers as possible. Thus, for each subcarrier, we use the uninterfered pilot symbols to estimate the h_{ref}, and we define them as set P_c. And then we can estimate $\widehat{h}_{ref}$ using the least square method:

$$\widehat{h}_{ref} = \arg \min_{h_{ref}} \sum_{n \in P_c} \left\| y_b(n) - h_{ref} \cdot x(n) \cdot e^{j\phi_t(n)} \cdot e^{jf_c n} \right\|_2^2, \quad (5)$$

where $\|\|_2$ is the l_2 norm. We assume that $\widehat{h}_{ref}$ remains stable for the time of a packet so that we can use it for all the data symbols in the backscatter packet.

C. Voting demodulation

Now, our goal is to demodulate data symbols using the established reference channel state information and phase errors. However, the interference from ambient signals is unpredictable, both in timing and spectrum, and we currently have information only on the uninterfered subcarriers of the pilot symbols. Including interfered subcarriers in our demodulation process could distort the signal phase, impacting accuracy. Building on previous efforts, we've identified all uninterfered subcarriers within pilot symbols (collectively referred to as set K) and calculated their reference channel state information h_{ref} and phase errors for all symbols. To extend this approach to data symbols, we introduce a voting-based demodulation algorithm.

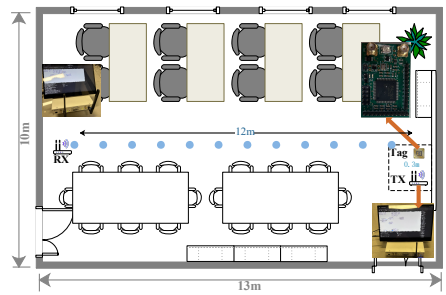


Fig. 8: Experiment environment in office.

For each subcarrier in n^{th} symbol, we now can estimate its phase:

$$\widehat{\phi}_t(n) = \frac{y_b(n)}{\widehat{h}_{ref}(n)x(n)e^{j\widehat{\phi}_{ce}(n)}} = \phi_t(n) \cdot \eta(n)$$

, where ϕ_t is the tag phase that has only two values, '0' and 'π', $\eta(n)$ is the random phase error that may caused by the interference and noise. In general, the noise would cause little impact on the tag phase, and the interference signal would largely influence the tag phase and even change it to another value. Thus, we first classify them according to which point their phase is closer to '0' or 'π'. Then, we use the voting method to determine the tag phase of each subcarrier.

We also apply the windowing method from pilot subcarrier detection (§IV-A) to assess subcarriers based on phase consistency within a window. If most subcarriers in a window share the same phase, they're considered uninterfered. We also compare the results of adjacent symbols to verify stability, leveraging interference signals' spectrum tendency to remain consistent over time. The most common phase value among identified uninterfered subcarriers of a symbol is chosen as the final phase.

V. Implementation

We have developed a prototype of SlickScatter utilizing a backscatter tag with an FPGA controller, a standard WiFi network adapter, and USRPs. The implementation details are as follows:

Tags: SlickScatter operates on an open-source backscatter tag platform [23], primarily comprising an envelope detector, a Microsemi AGLN250 low-power FPGA, and an ADG902 RF reflective switch. The envelope detector achieves $0.35\mu s$ synchronization, aligning with our multi-tag synchronization needs. Furthermore, the total power consumption of these components is exceptionally low, at just $33\mu W$, as indicated by the 45nm power analysis tool [8]. The FPGA handles both modulation and control logic.

Ambient Excitations: For ambient excitation, we employ a ThinkPad laptop coupled with a USRP2944R device. We set the WiFi packet payload and transmission parameters

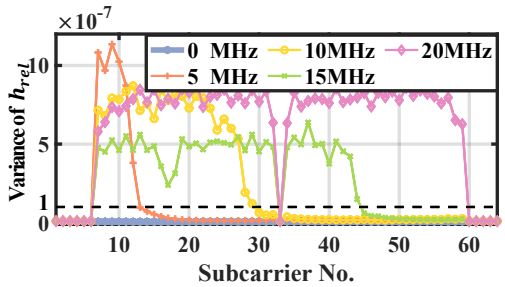
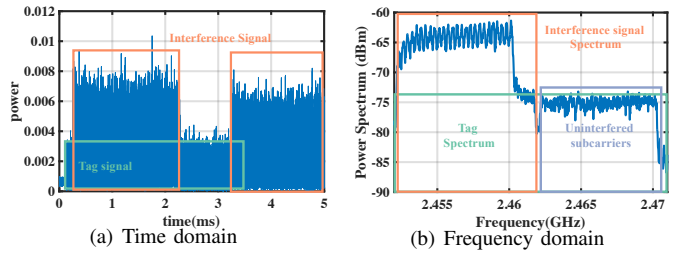


Fig. 9: The average variance of h_{rel} value for different interfered bandwidth.



(a) Time domain (b) Frequency domain

Fig. 10: An example of the interfered backscatter packet.

according to the 802.11g protocol. Subsequently, we generate the IQ signal using MATLAB and transmit the WiFi signal through the USRP2944R.

Receiver: The WiFi receiver is set up using a USRP-2944 [24] with an 802.11g PHY layer functioning as the receiver. Its dual boards operate as two RX access points (APs). The first board captures the ambient signal and regenerates the WiFi subcarriers. Meanwhile, the second board handles the reception and processing of backscatter signals, where subcarrier-level demodulation and interference cancellation are performed.

VI. Evaluation

In this section, we undertake a detailed performance evaluation of SlickScatter across various scenarios. The evaluation initiates with a comprehensive analysis focused on the system's efficiency in detecting uninterfered subcarriers and demodulating signals, which are achieved by generating controlled extra interference using USRP. After this foundational assessment, we further examine SlickScatter's performance in real-world environments to gauge its adaptability and effectiveness.

A. Micro benchmark

In this section, we aim to first identify unknown parameters, such as the threshold detailed in §IV-A and both the number of pilot symbols in one pilot group (defined as N_p) and the interval data symbols between two pilot groups (defined as N_d) in SlickScatter. Specifically, we select three pairs of N_p and N_d values to assess our algorithms' performance: $N_p = 2$ and $N_d = 38$, $N_p = 4$ and $N_d = 36$, and $N_p = 8$ and $N_d = 32$. These correspond to pilot symbol proportions of 5%, 10%, and 20%, respectively. Subsequently, we assess the efficacy of our algorithms for detecting uninterfered subcarriers and demodulating in this controlled environment.

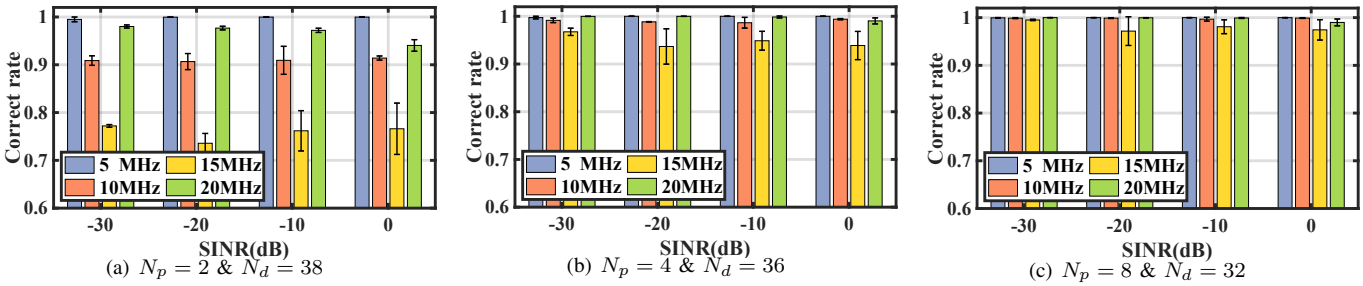
1) Experimental Setup: For the micro-benchmark experiments, we select a basement room without external interference sources, enabling precise control over the interference signal's strength and frequency using a USRP2944R. This setup allows for varying the signal-to-interference plus noise ratio (SINR) of the received signal and the number of interfered subcarriers, facilitating a comprehensive evaluation. Specifically, we adjust the SINR of the received signal from -30dB to 10dB and vary the interfered bandwidth across 5MHz, 10MHz, 15MHz, and 20MHz. The example of the interference bandwidth is 10MHz, and the SINR is about 15dB, as shown in Fig. 10. Consistent with prior studies [10], [18], we maintain a 0.3m distance between the TX and the tag and a 5m distance to the RX, achieving a received signal strength indicator (RSSI) of approximately -76dBm. We set the ambient WiFi packet length to 500 bits using the 802.11g protocol, corresponding to 158 symbols. We conduct experiments and receive 50000 packets with different interference bandwidths and SINR for the following evaluation.

2) Uninterfered Subcarriers Detection: We now first determine the threshold and then evaluate the performance of the uninterfered subcarrier detection algorithm with different N_p and N_d.

Threshold setting. We initiate our experiments by measuring the variance of h_{rel} to establish a threshold, as discussed in §IV-A, for distinguishing interfered and uninterfered subcarriers. We maintain $N_p = 4$ and $N_d = 36$, receiving signals at various SINR levels and different interference bandwidth and computing h_{rel} for each subcarrier. The average values from these packets are depicted in Fig. 9, demonstrating a clear distinction in h_{rel} values between interfered and uninterfered subcarriers. This distinction allows for their separation by establishing a threshold at 1×10^{-7} and $0.2 * M$ (where M represents the maximum h_{rel} value across all subcarriers).

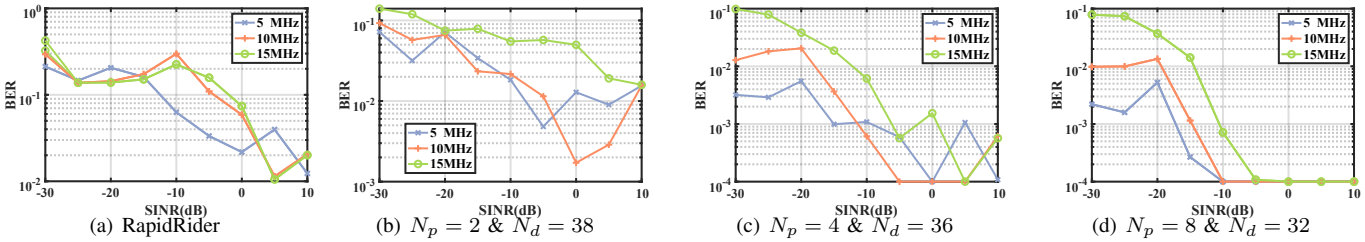
Detection performance. We then assess the effectiveness of our uninterfered subcarrier detection algorithm across various SINR levels and uninterfered bandwidths. The detection outcomes are categorized into three groups: equal to, greater than, and less than the actual number of uninterfered subcarriers. Detections exceeding the true count are considered false positives, which are deemed more impactful than detections falling short of the actual count (greater than 0), as the latter has a lesser effect on subsequent operations. Therefore, we classify results that are equal to or less than (greater than 0) the actual number of uninterfered subcarriers as correct judgments.

We conduct experiments to assess detection performance under conditions in which the interference signal strength matches or exceeds that of the backscatter signal. In scenarios where backscatter signal strength is lower than interference, the interference has a minimal impact on our demodulation,



(a) $N_p = 2$ & $N_d = 38$ (b) $N_p = 4$ & $N_d = 36$ (c) $N_p = 8$ & $N_d = 32$

Fig. 11: The performance of uninterfered subcarrier detection via different SINR, interfered bandwidth, and pilot symbol proportions.



(a) RapidRider (b) $N_p = 2$ & $N_d = 38$ (c) $N_p = 4$ & $N_d = 36$ (d) $N_p = 8$ & $N_d = 32$

Fig. 12: The performance comparison of tag data demodulation via different SINR and interfered bandwidth.

which is outside our evaluation focus. The correctness rates are displayed in Fig. 11, using three combinations of h_p and h_d to gauge detection efficacy. The results indicate that detection accuracy exceeds 0.9 for $N_p = 4$ or $N_p = 8$ across various interference bands and SINR levels. It is because the more pilot symbols provided, the greater the probability of change caused by interference, causing a higher variance value of h_{rel}. Notably, SINR has a negligible effect on detection accuracy above 0dB, demonstrating our method's capability to identify uninterfered subcarriers amidst strong interference accurately. Additionally, the interference band would also impact the accuracy of the detection but can also reach more than 0.92 when $N_p = 4$ or $N_p = 8$.

3) Demodulation: We proceed to evaluate our demodulation algorithm by conducting experiments with three pairs of h_p and h_d values to assess performance. The bit error rate (BER) across different SINR levels and uninterfered bandwidths is calculated to measure the effectiveness of our demodulation. Additionally, we benchmark our algorithm against RapidRider [19], which assigns one tag bit per symbol and uses a symbol-level codeword translation method for demodulation.

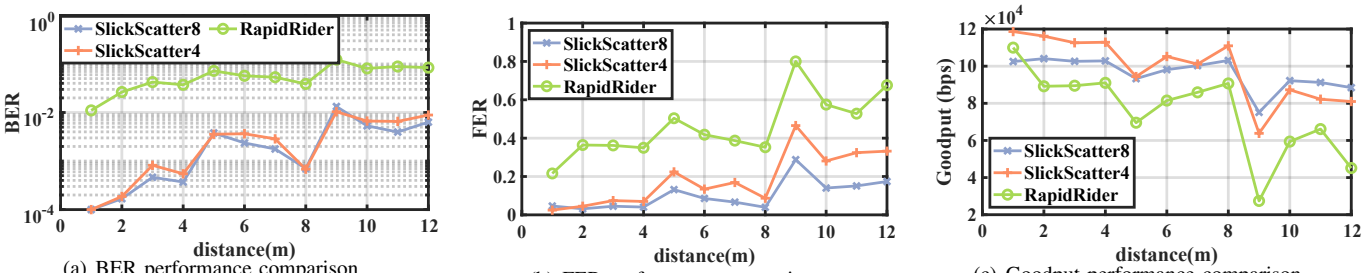
Results shown in Fig. 12 reveal that with increasing SINR, BER decreases accordingly. Our algorithm maintains BER below 0.1 even in the presence of interference, outperforming RapidRider, which achieves similar BER levels only when SINR exceeds -5 dB. Notably, at SINR levels below -10 dB, our algorithm reduces BER to two orders of magnitude lower than that of RapidRider. We can also see that an increased N_p symbol proportions significantly reduces BER, where the 20% pilot symbols can reduce the BER under 10^{-3} when SINR is higher than -15 dB. This is because the more pilot symbols provided, the more accurate CPE and CSI estimates will be. Furthermore, while a broader interference bandwidth increases BER, our demodulation technique manages to keep it lower than RapidRider. In essence, keeping the pilot symbol ratio at 10% and 20% lowers BER with less detriment to overall system performance. For the following analysis, we will engage two specific configurations, **SlickScatter4** ($N_p = 4, N_d = 36$) and **SlickScatter8** ($N_p = 8, N_d = 32$), in our forthcoming experiments.

B. Macro benchmark

In this section, we mainly evaluate the performance of SlickScatter in a real-world environment (shown in Fig. 8).

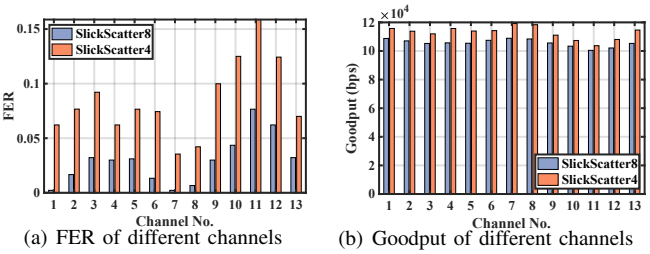
1) Experiment setup: For our experiments, we select an office environment characterized by prevalent unknown interference. We assess SlickScatter's performance across various conditions. These conditions include variations in the tag-RX distance, the communication channel in the 2.4GHz frequency band, and different times of day. We ensure the transmission of WiFi packets at a consistent 500 bits, receiving 900 backscatter packets for each scenario. Additionally, we maintain a 0.3m distance between the TX and tag throughout the experiments.

2) Performance with distance: Initially, we assess SlickScatter's performance over varying distances between the tag and the RX, adjusting the distance from 1m to 12m (the office length) in 1m increments, receiving backscatter packets in channel NO.9, and demodulating them using SlickScatter4 and SlickScatter8 settings. We then calculate the BER, frame error rate (FER), and goodput for each



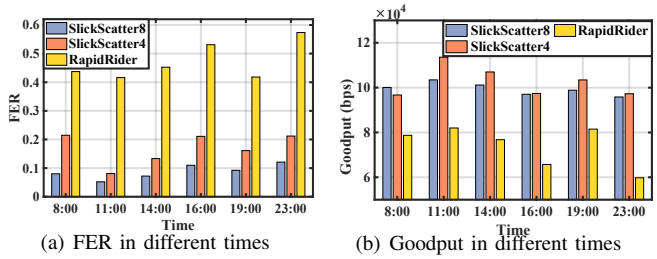
(a) BER performance comparison (b) FER performance comparison (c) Goodput performance comparison

Fig. 13: The performance comparison of indoor scenario



(a) FER of different channels (b) Goodput of different channels

Fig. 14: The performance in different channels of 2.4GHz band.



(a) FER in different times (b) Goodput in different times

Fig. 15: The performance at different times of one day.

setup and compare these metrics with RapidRider to gauge performance. Results in Fig. 13 demonstrate that SlickScatter consistently achieves lower BER and FER and higher goodput than RapidRider across most distances. This means that the sacrifice of data bits for setting pilot bits would improve rather than reduce the system performance when interference happens. Remarkably, even at a 12m distance, SlickScatter maintains a BER of about 10^{-2} and a FER of about 0.2, indicating its suitability for indoor environments and compared to RapidRider SlickScatter8 can reduce the frame rate by 50% and reach 1.96× goodput. Moreover, comparing pilot symbol proportions, SlickScatter8 consistently shows lower BER and FER, while SlickScatter4 exhibits higher goodput in most scenarios. The results show that our work can break through the restrictions of unknown interference and support more transmission opportunities for backscatter tag in the commercial WiFi band.

3) Performance in different channels: We configure the backscatter tag to operate across 2.4 GHz WiFi channels (No.1-13), applying different frequency shifts to test against unknown interference, as detailed in Fig. 14. Evaluating performance, we record the FER and goodputs at a 0.3 m distance from TX to tag and a 6 m distance from tag to RX, with outcomes presented in Fig. 14. The results indicate that FER almost remains below 0.15 across all scenarios, with average goodputs reaching approximately 100kbps for both SlickScatter8 and SlickScatter4 settings. Notably, on channels No. 7 and No. 8, our system achieves a FER below 0.05, demonstrating optimal performance on these channels.

4) Performance at different times: We also conduct experiments at different times of one day to evaluate the performance of SlickScatter at different times. We also measure the FER and goodputs of these times under 0.3 m TX to tag distance, 6 m tag to RX distance in channel No.9 and illustrate the FER and goodput results in Fig. 15 compared with RapidRider. We can see the FERs of SlickScatter8 always can keep lower than 0.15 while the FER value of RapidRider is always more than 0.4. Furthermore, our system's goodput approaches 100kbps in most periods, consistently surpassing that of RapidRider.

VII. Related Work

WiFi backscatter systems have garnered substantial attention for embedding tag data using ambient WiFi signals [4], [8]–[14], [17]–[19], [25]. Notable methodologies like codeword translation, pioneered by HitchHike [8] and employed by successors just like FreeRider [9], MOXScatter [10] and TiScatter [18] offer adaptability to different WiFi protocols. In order to be more suitable for the actual environmental signal situation, RapidRider [19] realize efficient WiFi backscatter with uncontrolled ambient signals, CAB [17], Leggiero [13] and Twoferscatter [26] realize the backscatter system without knowing the content of ambient WiFi signal. To optimizing the performance of WiFi backscatter communication, VMScatter [11] and SyncScatter [12] propose the virtual MIMO and synchronization mechanism to improve the throughput of WiFi backscatter. XSHIFT [27] propose a true tag to decrease the power consumption of the prototype tag to tens of μW. SubScatter [14] and TScatter [20] propose novel modulation schemes to improve the throughput of WiFi backscatter.

However, these systems do not consider the interference of unknown ambient signals, which often happens in the actual environment.

VIII. Discussion

In our system, despite the inability to demodulate signal when its entire spectrum is interfered with, we significantly enhance the WiFi backscatter system's robustness and effectiveness. This improvement comes from the low likelihood of such complete spectrum interference occurring. Additionally, the consistency of data transmission is less critical in the context of low-power devices, which are typically not used for transmitting large or highly accurate data streams. Therefore, we can tolerate a limited amount of packet loss.

IX. Conclusion

In this paper, we propose SlickScatter, which can retrieve the WiFi backscatter signal from the interference signal. We introduce the pilot symbols into the backscatter packets and propose the uninterfered subcarriers detection algorithm, phase fitting algorithm to estimate CPE and CSI, and vote algorithm to demodulate the backscatter bits. We prototype SlickScatter and evaluate it in a real-world environment. The results show that SlickScatter can retrieve the WiFi backscatter signal from the interference signal with higher robustness and accuracy than previous works.

X. Acknowledgments

The research is partially supported by National Key R&D Program of China No. 2021ZD0110400, NSFC with No. 62072424, U20A20181, Key Research Program of Frontier Sciences, CAS. No. ZDBS-LY-JSC001, Hefei Municipal Natural Science Foundation with No. 2022016, the Fundamental Research Funds for the Central Universities with No. WK3500000008.

References

[1] Y. Peng, L. Shangguan, Y. Hu, Y. Qian, X. Lin, X. Chen, D. Fang, and K. Jamieson, "PLoRa: A passive long-range data network from ambient LoRa transmissions," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, 2018, pp. 147–160.

[2] Z. Chi, X. Liu, W. Wang, Y. Yao, and T. Zhu, "Leveraging ambient lte traffic for ubiquitous passive communication," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 172–185.

[3] Y. Feng, S. Chen, W. Xi, S. Wang, J. Zhao, and W. Gong, "Heartbeating with lte networks for ambient backscatter," *IEEE Transactions on Mobile Computing*, 2023.

[4] F. Dehbashi, A. Abedi, T. Brecht, and O. Abari, "Verification: can wifi backscatter replace rfid?" in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, 2021, pp. 97–107.

[5] V. Liu, A. Parks, V. Talla, S. Gollakota, D. Wetherall, and J. R. Smith, "Ambient backscatter: Wireless communication out of thin air," *ACM SIGCOMM computer communication review*, vol. 43, no. 4, pp. 39–50, 2013.

[6] M. H. Mazaheri, A. Chen, and O. Abari, "Mmtag: A millimeter wave backscatter network," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, 2021, pp. 463–474.

[7] N. Van Huynh, D. T. Hoang, X. Lu, D. Niyato, P. Wang, and D. I. Kim, "Ambient backscatter communications: A contemporary survey," *IEEE Communications surveys & tutorials*, vol. 20, no. 4, pp. 2889–2922, 2018.

[8] P. Zhang, D. Bharadia, K. Joshi, and S. Katti, "HitchHike: Practical Backscatter Using Commodity WiFi," in *Proceedings of the 14th ACM Conference on Embedded Network Sensor Systems CD-ROM*, 2016, pp. 259–271.

[9] P. Zhang, C. Josephson, D. Bharadia, and S. Katti, "FreeRider: Backscatter Communication Using Commodity Radios," in *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies*, 2017, pp. 389–401.

[10] J. Zhao, W. Gong, and J. Liu, "Spatial stream backscatter using commodity wifi," in *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services*, 2018, pp. 191–203.

[11] X. Liu, Z. Chi, W. Wang, Y. Yao, and T. Zhu, "VMscatter: A versatile MIMO backscatter," in *17th USENIX Symposium on Networked Systems Design and Implementation*, 2020, pp. 895–909.

[12] M. Dunna, M. Meng, P.-H. Wang, C. Zhang, P. Mercier, and D. Bharadia, "SyncScatter: Enabling WiFi like synchronization and range for WiFi backscatter Communication," in *18th USENIX Symposium on Networked Systems Design and Implementation*, 2021, pp. 923–937.

[13] X. Na, X. Guo, Z. Yu, J. Zhang, Y. He, and Y. Liu, "Leggiero: Analog wifi backscatter with payload transparency," in *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*, 2023, pp. 436–449.

[14] J. Yu, C. Du, J. Liu, R. Zhang, and S. Wang, "Subscatter: Subcarrier-level ofdm backscatter," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.

[15] S. Wang, Y. Yan, Y. Chen, P. Yang, and X.-Y. Li, "Spray: A spectrum-efficient and agile concurrent backscatter system," *ACM Transactions on Sensor Networks*, vol. 20, no. 2, pp. 1–21, 2024.

[16] A. Abedi, F. Dehbashi, M. H. Mazaheri, O. Abari, and T. Brecht, "Witag: Seamless wifi backscatter communication," in *the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 240–252.

[17] Y. Yang, L. Yuan, J. Zhao, and W. Gong, "Content-agnostic backscatter from thin air," in *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*, 2022, pp. 343–356.

[18] C. Du, J. Liu, S. Wang, R. Zhang, W. Gong, and J. Yu, "Timespan-based backscatter using a single cots receiver," in *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*, 2023, pp. 450–461.

[19] Q. Wang, S. Chen, J. Zhao, and W. Gong, "Rapidrider: Efficient wifi backscatter with uncontrolled ambient signals," in *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 2021, pp. 1–10.

[20] X. Liu, Z. Chi, W. Wang, Y. Yao, P. Hao, and T. Zhu, "Verification and redesign of OFDM backscatter," in *18th USENIX symposium on networked systems design and implementation*, 2021, pp. 939–953.

[21] J. Terry and J. Heiskala, *OFDM wireless LANs: A theoretical and practical guide*. Sams publishing, 2002.

[22] Q. Wang, J. Yu, C. Xiong, J. Zhao, S. Chen, R. Zhang, and W. Gong, "Efficient backscatter with ambient wifi for live streaming," in *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*, 2020, pp. 1–6.

[23] C. Xu and P. Zhang, "Open-source software and hardware platforms for building backscatter systems," *GetMobile: Mobile Computing and Communications*, vol. 23, no. 1, pp. 16–20, 2019.

[24] N. INSTRUMENTS, "Ni usrp-2944," 2024. [Online]. Available: https://www.ni.com/zh-cn/support/model.usrp-2944.html

[25] X. He, W. Jiang, M. Cheng, X. Zhou, P. Yang, and B. Kurkoski, "Guardrider: Reliable wifi backscatter using reed-solomon codes with qos guarantee," in *2020 IEEE/ACM 28th International Symposium on Quality of Service (IWQoS)*. IEEE, 2020, pp. 1–10.

[26] Y. Yang, W. Gong, and Y. Yang, "Ambient backscatter with a single commodity ap," in *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, 2023, pp. 1–10.

[27] M. Rostami, K. Sundaresan, E. Chai, S. Rangarajan, and D. Ganesan, "Redefining passive in backscattering with commodity devices," in *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*, 2020, pp. 1–13.